ALA 1 Report: Cloud Web Application Deployment

Name: Jainisha Makwana **EnRoll No:** 20230905090017 **Class:** c

Submitted to: Dhaval Chandarana

1. Objective

The objective of this activity is to create and deploy a basic web application on a cloud platform, configure a compute service and a storage service, make the application accessible over the internet, and demonstrate basic resource management.

2. Cloud Services Used

Requirement	Service used	Purpose
Compute service (app service)	Render Web Service (Free instance)	Runs the Flask web application using Gunicorn
Storage service (database)	Neon PostgreSQL (Free plan)	Stores the notes entered by users permanently
Source code hosting	GitHub	Stores the code; Render deploys from it

Render is a cloud platform that provides an app service (PaaS). The platform manages the server, so only the application code needs to be uploaded. **Neon** is a managed cloud PostgreSQL database service. Keeping the database separate from the application means the data stays safe even when the application restarts.

3. Application Description

The application, **Cloud Notes**, is a small web application built with Python and Flask. A visitor enters a name and a message. The message is saved in the cloud database and shown in a table on the same page. After a page refresh the notes are still visible, which proves that the data is stored in the database and not in the page.

4. Architecture

User's browser (internet) -> Render Web Service (Flask + Gunicorn) -> Neon PostgreSQL database

The source code is stored in a GitHub repository, and Render builds and deploys the application from this repository.

5. Deployment Details

Field	Details
Cloud platform (application)	Render (render.com)

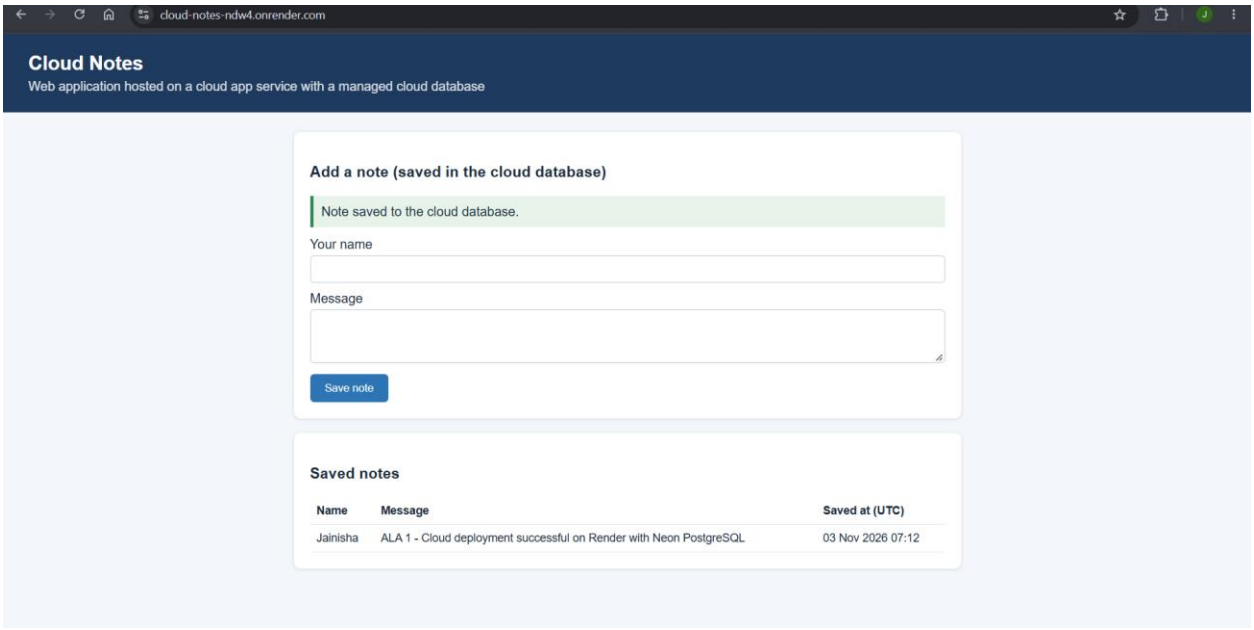
Field	Details
Cloud platform (database)	Neon (neon.com), AWS Asia Pacific Singapore (ap-southeast-1)
Compute service	Render Web Service, Free instance, Python 3, Gunicorn
Storage service	Neon PostgreSQL, Free plan
Region of web service	Singapore
Live application URL	https://cloud-notes-ndw4.onrender.com
Source code (GitHub)	https://github.com/Jai-ni-sha/cloud-notes
Build command	pip install -r requirements.txt
Start command	gunicorn app:app
Configuration	Environment variables DATABASE_URL and SECRET_KEY, kept outside the code
Technology	Python 3, Flask, Gunicorn, PostgreSQL

6. Deployment Steps

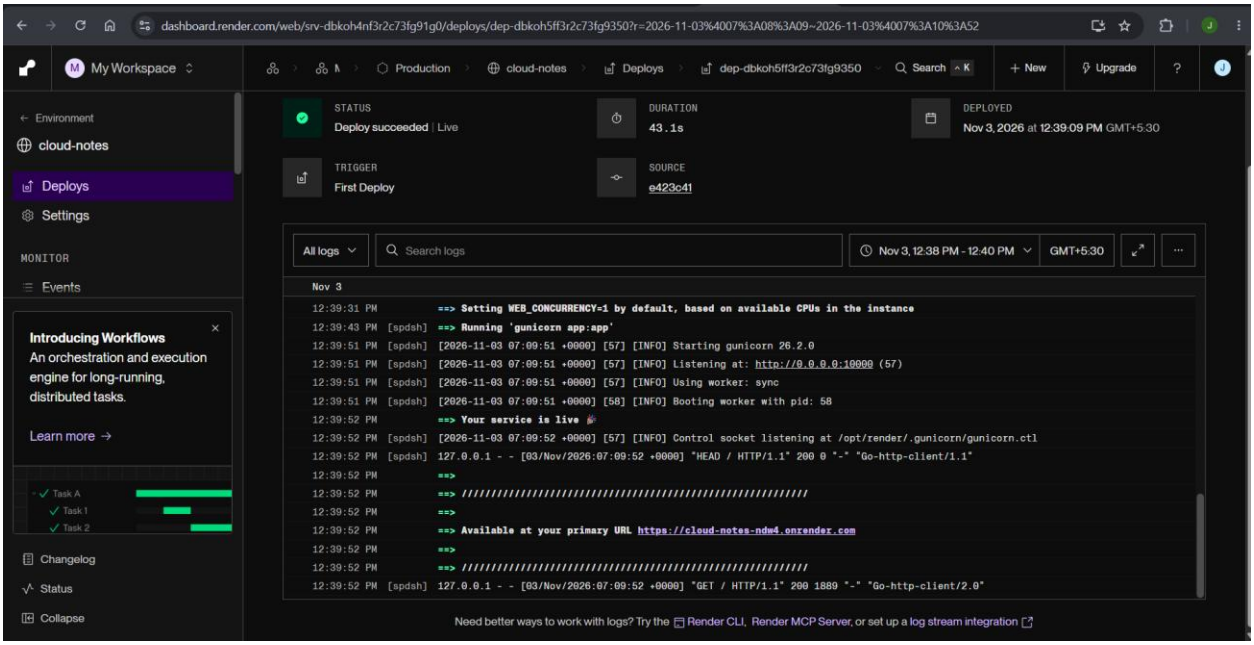
1. Wrote the Flask application (app.py) and listed its libraries in requirements.txt.
2. Created a public GitHub repository named cloud-notes and uploaded app.py, requirements.txt and README.md.
3. Created a Neon account and a PostgreSQL project in the Singapore region, and copied its connection string.
4. Created a Render Web Service connected to the GitHub repository, with build command pip install -r requirements.txt and start command gunicorn app:app.
5. Selected the Free instance type and added the environment variables DATABASE_URL (Neon connection string) and SECRET_KEY.
6. Deployed the service. Render built the application and the status changed to Live.
7. Opened the public URL in a browser, added notes, and refreshed the page to confirm that the notes are stored in the database.

7. Screenshots

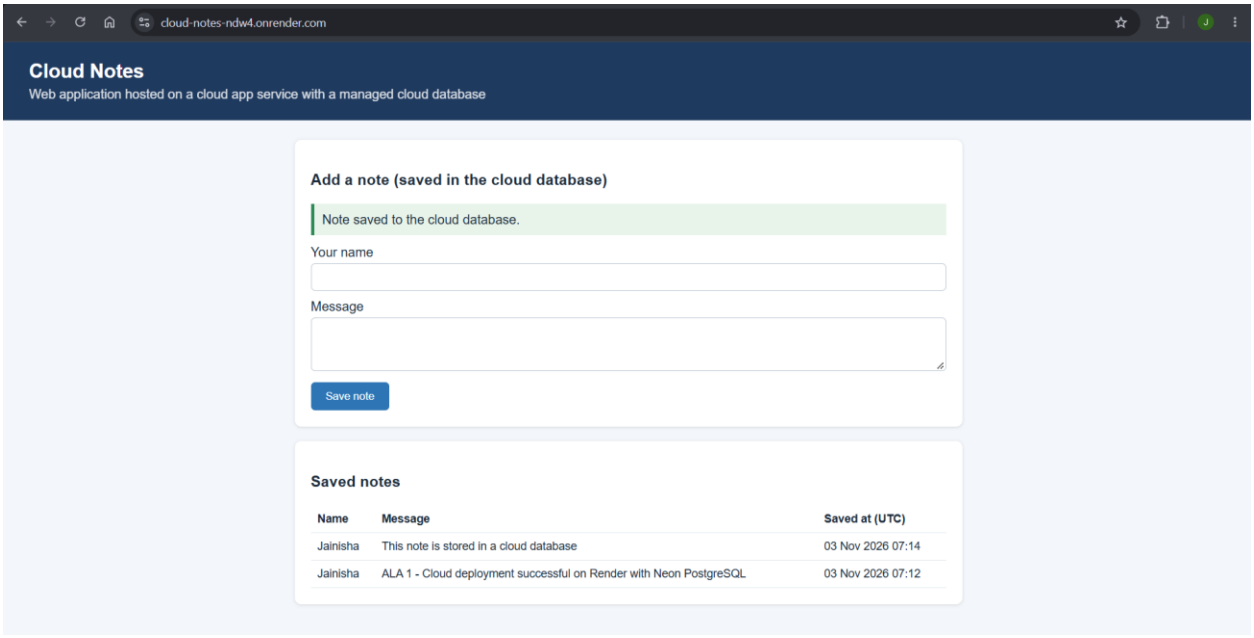
Screenshot 1: Running application with saved notes



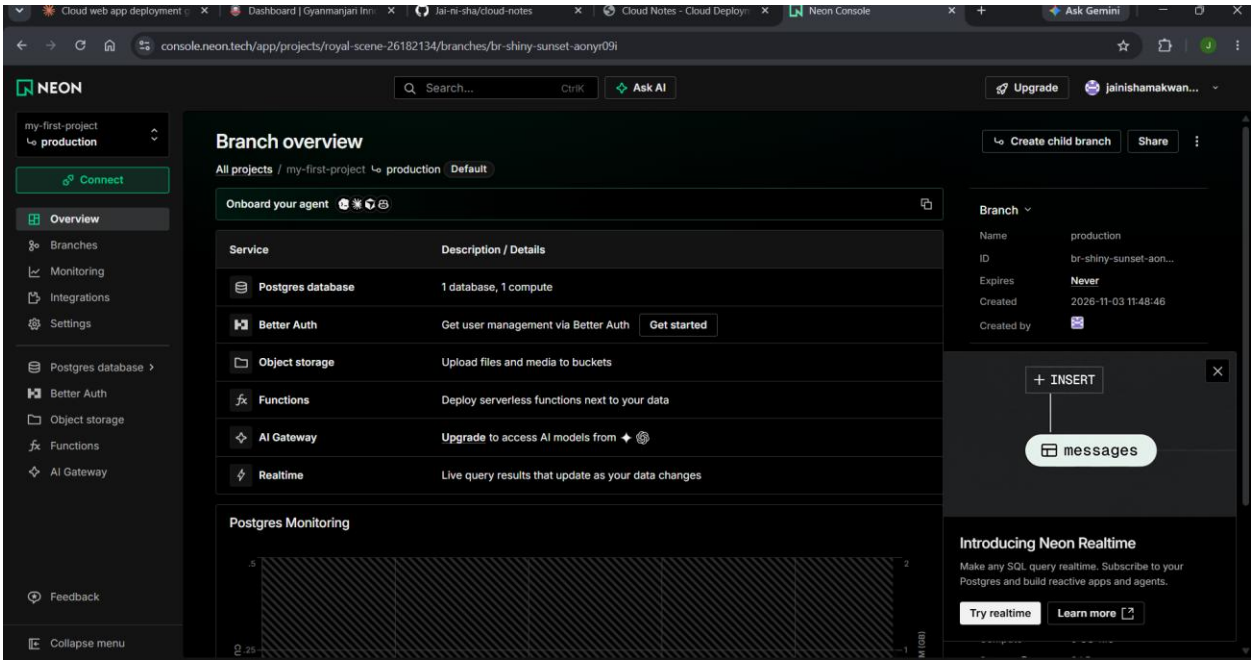
Screenshot 2: Render dashboard showing the deployment is Live



Screenshot 3: Running application with saved notes



Screenshot 4: Neon database dashboard



8. Resource Management

- **Instance selection:** The smallest Free instance type was chosen to avoid cost.
- **Monitoring:** Application logs and metrics were checked in the Render dashboard.
- **Configuration management:** The database link and secret key are stored as environment variables, not in the source code.

- **Continuous deployment:** The service is connected to GitHub, so updated code can be redeployed.
- **Database management:** The Neon dashboard shows the project, branch and compute, and the production branch is set to never expire.
- **Cost control:** Both services run on free plans. When the work is no longer needed, the Render service and the Neon project can be suspended or deleted.

9. Challenges Faced and Solutions

Challenge	Solution
When the application was run on my laptop it showed a database connection error	The application needs a cloud database. This was expected on the laptop, and it worked after connecting to the Neon database on Render
Render's free database expires after about 30 days	Used Neon PostgreSQL, whose free plan does not have this time limit
The free web service sleeps after inactivity	The first page load can take about one minute; this is a limitation of the free plan

10. Conclusion

In this activity I deployed a Flask web application on Render's web service and used a managed Neon PostgreSQL database for storage. The application is accessible over the internet through a public URL. I managed resources using logs, metrics, environment variables, free plans and GitHub-based deployment. This activity helped me understand PaaS, the separation of compute and storage, and the pay-as-you-go model of cloud computing.